Handling Mixed Variables

Splitting columns that carry two kinds of information

Contents

Handling Mixed Variables 2
33.1 Two different patterns of mixture 2
33.2 Why standard encoders fail 3
33.3 The splitting strategy 3
33.4 The Ticket column, traced 3
33.5 The Number column, traced 4
33.6 Rebuilding the pipeline 5
33.7 Cardinality after the split 5
33.8 Diagnostics and common mistakes 6
33.9 Recap, practice and sources 6

CHAPTER 33

Handling Mixed Variables

Splitting columns that carry two kinds of information

WHAT THIS CHAPTER BUILDS

Mixed variables break the usual contract that a column is either numeric or categorical. This chapter separates the two patterns of mixture, works through the Titanic Cabin and Ticket columns line by line, and rebuilds them as clean numeric and categorical features without losing information.

33.1 Two different patterns of mixture

A variable is mixed when a single column mixes numeric and categorical information. There are two structurally different versions of this problem, and they need different repairs.

Mixed within each cell. Every value contains both a text token and a number. The Titanic `Cabin` column holds strings such as `B5`, `C123`, `E46`; the letter encodes a deck or section, the number encodes a cabin identifier. The `Ticket` column holds values such as `PC 17599`, `STON/O2. 3101282`, `A/5 21171`, and plain numbers such as `24160`. Every cell is one token that means two things at once.

Mixed across rows. Some cells contain numbers and other cells contain words. The lecture demonstrates this with a column called `Number` in which some rows hold integers and other rows hold the string `alone`. No single cell is ambiguous; the column as a whole is ambiguous.

The distinction matters because the repair differs. For within-cell mixture you split each cell into parts. For across-row mixture you separate the rows into two types and let each type use its own feature.

Common Examples of Mixed Variables

Height	`5 feet 7 inches`	*Numeric + Unit category*
Weight	`70.5 kg`	*Numeric + Unit category*
Date-Time	`2024-01-15 14:30:00`	*Numeric parts + separators*
Coordinates	`40.7128° N, 74.0060° W`	*Numeric + Direction category*
File Size	`2.5 MB`	*Numeric + Unit category*
Range	`10-20`	*Two numerics + separator*

Two patterns of mixture. On the left every Cabin cell contains a letter and a number; on the right the Number column holds integers in some rows and the word alone in others.

Mixed is not the same as missing

> A `NaN` means the value is absent. A mixed value is present but overloaded. If you drop the row because the column is "not numeric", you throw away a cabin or a ticket that still carries information.

33.2 Why standard encoders fail

A standard preprocessing pipeline assumes each column has one type. A mixed column violates that assumption, and the failure appears in different places depending on which way you push it.

Cast the column to numeric with `pd.to_numeric(errors="coerce")` and every text cell becomes `NaN`. The cabin letters disappear, and the model now believes the value is missing. Cast it to string and every number becomes a category: cabin numbers 1, 2, …, 100 become 100 separate levels, which explodes the one-hot matrix and destroys the ordering that the numbers had.

`OneHotEncoder` on the raw column produces one level per unique string. Titanic has hundreds of distinct tickets. With 891 rows, a one-hot vector of that width memorises the training data instead of learning a pattern, and unseen test tickets arrive as all-zero rows. The encoder is doing exactly what was asked; the input representation is wrong.

The correct response is to change the representation before the encoder sees the column.

33.3 The splitting strategy

The repair has three rules. First, never destroy the original column; keep it until the new features are validated. Second, split the token into its numeric part and its categorical part, using `str` accessors and regular expressions. Third, treat the two new columns as ordinary features with their own types, their own imputers, and their own encoding.

```
import numpy as np
import pandas as pd

df["cabin_raw"] = df["Cabin"]                 # keep the original
df["cabin_deck"] = df["Cabin"].str[0]         # categorical: A, B, C, ...
df["cabin_no"] = pd.to_numeric(
    df["Cabin"].str.extract(r"(\d+)")[0],     # numeric: 5, 123, 46, ...
    errors="coerce",
)
```

`str.extract(r"(\d+)")[0]` pulls the first group of digits out of each cell and returns a Series; `[0]` selects the column of the resulting DataFrame. `pd.to_numeric(..., errors="coerce")` converts the token to a number and leaves `NaN` wherever there was no digit. The letter part becomes a small, well-behaved categorical feature; the number part stays numeric. Both parts keep their own `NaN` for cells where that part does not exist.

33.4 The Ticket column, traced

Tickets are messier than cabins. The prefix and the number are separated by spaces, slashes, or periods, and some tickets have no prefix at all. The extraction below handles the common forms.

```
t = df["Ticket"].astype(str).str.strip()

df["ticket_numeric"] = pd.to_numeric(
    t.str.extract(r"(\d+)$")[0], errors="coerce"
```

```
)
prefix = (
    t.str.replace(r"\d+", "", regex=True)     # remove digits
     .str.replace(r"[\s./]+", "", regex=True) # remove separators
     .str.strip()
)
df["ticket_prefix"] = prefix.replace("", np.nan)
```

The numeric part takes the trailing digits. The prefix is what remains after digits and separators are deleted: `PC`, `STONO2`, `A5`, `PP`. An empty result becomes `NaN`, which means "no prefix", not "unknown prefix"; if that distinction matters to the model, replace it with a dedicated level such as `NO_PREFIX` instead.

Notice that both new columns can be missing. `ticket_numeric` is `NaN` when the ticket has no digits; `ticket_prefix` is `NaN` when there is no text. Each missing value means something different, and each imputer must be fitted on the training split only.

> **The Deck feature is a compression**
>
> Collapsing `B5` and `B123` into deck `B` reduces hundreds of cabin values to a handful of decks. That is a deliberate loss: the model gets a coarser feature with more support per level. Keep the raw cabin number if the exact location matters and the sample is large enough to estimate it.

33.5 The Number column, traced

The across-row pattern needs a different tool. Split the column by asking which cells can be read as numbers.

```
num = pd.to_numeric(df["Number"], errors="coerce")

df["Number_num"] = num                      # numeric rows keep their value
df["Number_cat"] = df["Number"].where(
    num.isna(), np.nan
)                                           # words keep their text
df["Number_cat"] = df["Number_cat"].fillna("NO_NUMBER")
df["Number_num"] = df["Number_num"].fillna(0)
```

The `where` clause is the whole trick: the categorical part keeps the original text only where the numeric conversion failed (the word `alone`), and becomes `NaN` where the cell was a number. The numeric part does the opposite; rows that held `alone` have no number and become `NaN`.

The two fills are domain decisions, not defaults. `0` in the numeric part does not mean "the count is zero" unless zero is a legitimate value; here `alone` plausibly means the count is none, so `0` is defensible. `NO_NUMBER` in the categorical part marks a real category. If the interpretation is unknown, leave the `NaN` and let a median or mode imputer handle it, and add a missing indicator so the model can see the absence.

Pattern	Split into	Imputation
Cabin `B5`	`cabin_deck` = B, `cabin_no` = 5	Mode for deck, median for number
Ticket `PC 17599`	`ticket_prefix` = PC, `ticket_numeric` = 17599	Mode or `NO_PREFIX`, median
Number `alone` vs `5`	`Number_cat` = NO_NUMBER, `Number_num` = 5	Domain fill, or imputer plus indicator

33.6 Rebuilding the pipeline

Once the column is split, the pieces go into ordinary branches of a `ColumnTransformer`. The numeric branch gets an imputer and any scaling; the categorical branch gets an imputer and one-hot encoding. Nothing about the mixed column is special any more.

```
from sklearn.compose import ColumnTransformer
from sklearn.impute import SimpleImputer
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import OneHotEncoder

numeric_features = ["cabin_no", "ticket_numeric", "Number_num"]
categorical_features = ["cabin_deck", "ticket_prefix", "Number_cat"]

numeric_pipe = Pipeline([("impute", SimpleImputer(strategy="median"))])
categorical_pipe = Pipeline([
    ("impute", SimpleImputer(strategy="most_frequent")),
    ("onehot", OneHotEncoder(handle_unknown="ignore")),
])

preprocess = ColumnTransformer([
    ("num", numeric_pipe, numeric_features),
    ("cat", categorical_pipe, categorical_features),
])
```

`handle_unknown="ignore"` matters for tickets: a prefix that never appears in training arrives in production, and the encoder must produce zeros rather than raise an error. Fit the whole pipeline on `X_train` only; the extraction itself is stateless, but the imputers and the one-hot vocabulary are not.

33.7 Cardinality after the split

Splitting usually reduces cardinality, but not always. Cabin decks collapse to a few levels. Ticket prefixes collapse to a few dozen. Ticket numbers, however, stay as numerous as the tickets, and a prefix such as `NULL` or `NO` may still dominate the categorical branch.

When a categorical part remains high-cardinality, the options are to group rare levels, use frequency or target encoding, or hash the string into a fixed number of bins. Each option trades information for a bounded vocabulary. The lecture's focus is the split itself; these are follow-on tools for the columns that stay wide after splitting.

Before Splitting

Height_Mixed
5 feet 7 inches
6 feet 2 inches
5 feet 1 inch

After Feature Splitting

Height_Value	Height_Unit
5, 7	feet, inches
6, 2	feet, inches
5, 1	feet, inches

The mixed column is decomposed into a categorical part and a numeric part, which then enter separate pipeline branches with their own imputation and encoding.

33.8 Diagnostics and common mistakes

The first check is whether the split actually recovered the structure. Run `value_counts` on the categorical part and a histogram on the numeric part. If the numeric part is mostly `NaN`, the regular expression did not match the real format. If the categorical part has one level per row, the split did not compress anything.

The second check is leakage. The extraction is deterministic, but imputers, encoders, and any level grouping must be fitted on training data. Grouping rare levels by frequency computed on the full dataset leaks test distribution into training.

The third check is alignment. The two parts must stay row-aligned after any filtering. A common bug is to extract from the original `df` after rows have been dropped from a transformed copy.

Common mistakes:

- Dropping the original column immediately instead of validating the new features first.
- Converting the whole column to numeric and treating the text cells as missing.
- One-hot encoding raw ticket strings and wondering why the model memorises the training set.
- Filling `alone` with a numeric value that does not mean the same thing as a real count.
- Fitting the categorical imputer before the train/test split.

33.9 Recap, practice and sources

The chain is: a mixed column either contains two tokens in each cell or holds different types in different rows; split within-cell mixtures with `str` extract and `to_numeric`; separate across-row mixtures by asking which cells convert; keep the original; give the numeric and categorical parts their own imputation and encoding inside one `ColumnTransformer`; validate the split before trusting the model.

> **Lock-in line**
>
> The encoder cannot fix a column whose type is ambiguous. Ambiguity is resolved by splitting the representation, not by choosing a more permissive encoder.

Practice

1. Apply the Cabin split to Titanic. How many decks and how many distinct cabin numbers result?
2. Extract the Ticket prefix with the regex above. Which tickets end up with an empty prefix?
3. Build the `Number` split with `alone` and integers. Show `value_counts` for both parts.

4. Put the split features into a `ColumnTransformer` and compare the model against the raw-string baseline.
5. Add a rare-level grouping step driven by training-set frequencies only. Verify that test-set-only levels are handled.

Sources and scope

Video 33, “Handling Mixed Variables”, CampusX 100 Days of Machine Learning. The lecture walks through Titanic Cabin and Ticket columns and the mixed Number column, splitting each into categorical and numeric parts. The regex variants and the rare-level options are supplementary examples consistent with the lecture’s approach.